

Distributed Traffic Monitoring with Fault-Tolerant Replication

Course/Cluster: Khoury College — Distributed Systems (CS 6650)

Project Team #23

Team Members: Che-Yi Wu, Yu-Hsuan Yang

1 Overview

- Build a **distributed real-time traffic-monitoring system** simulating smart-city infrastructure.
 - Many **sensor nodes** generate continuous data (**road**, **vehicle_count**, **avg_speed**).
 - Data is **sharded and replicated across multiple servers** for scalability and fault tolerance.
 - A **map-style dashboard** displays each road's live status:
 Congested  Moderate  Smooth.
 - Demonstrates **partitioning (hash sharding)**, **replication**, **consistency**, and **failure recovery**.
-

2 Goals

- Continuously collect and update traffic data.
 - Distribute load across servers using **hash sharding**.
 - Maintain consistent replicated data during failures.
 - Keep end-to-end latency < 1 s from sensor $\rightarrow$ dashboard.
 - Show quantitative metrics proving scalability and fault tolerance.
-

3 Detailed Design

- **Sensor Nodes**
 - Simulate intersections or vehicles; send JSON `{road,timestamp,vehicle_count,avg_speed}` via HTTP POST `/ingest`.
 - Randomly change roads to mimic vehicles moving.
 - **Server Cluster (3 nodes)**
 - **Hash sharding:** `shard_id = hash(road) % P` ($P \approx 6$).
 - Each shard has 1 leader + 1 follower (replication factor 2).
 - Round-robin placement → balanced load across srv-1/2/3.
 - Leader handles writes & replicates to follower; followers can serve reads.
 - **Replication Layer**
 - Lightweight Raft-style: heartbeats, append entries, majority commit, leader election on timeout.
 - **Dashboard UI**
 - Web map (e.g., HTML + Chart.js or Streamlit) polls `/summary` every few hundred ms.
 - Colors roads green/yellow/red based on latest average speed and vehicle count.
 - **Data Storage**
 - In-memory aggregates + optional JSON snapshot for persistence.
-

4 Assumptions

- **Failures:** Crash-stop; one server failure at a time.

- **Network:** Reliable TCP; minor delay tolerated.
- **Workload:** 10–200 sensors, 1–5 updates/s.
- **Cluster:** 3 servers, 6 shards, replication factor 2.
- **Users:** Dashboard viewers and evaluation scripts.

5 Modules

Module	Description
SensorNode	Generates ongoing traffic data and sends updates
MainServer	Handles HTTP ingest/read and routes events by shard
Replication	Leader–follower log replication + failover
Aggregator	Computes averages and classifies traffic color
Utils	Logging, timing, CLI helpers
Dashboard	Map UI visualizing red/yellow/green roads

6 Evaluation Plan

Functionality	Evaluation Method	Metric
Live updates	Observe dashboard color changes in real time	< 1 s delay

Hash sharding	Log distribution of roads per server	Balanced load ($\approx \frac{1}{3}$ each)
Replication consistency	Compare leader/follower indices	≤ 1 entry difference
Failover	Kill leader; measure recovery	≤ 3 s re-election
Scalability	Increase sensors 10 $\rightarrow$ 100	Throughput vs load curve
Accuracy	Check computed speed vs ground truth	≥ 99 % correctness

7 Running on Khoury Linux Cluster

- **Language:** C++11 **Compiler:** g++
- **Dependencies:** `nlohmann/json.hpp`, `cpp-httplib/httpplib.h` (header-only)

Makefile snippet

```
CXX = g++
CXXFLAGS = -Wall -std=c++11 -pthread
```

Run example

```
make -j

./bin/server --id srv1 --http 5000 --peers
127.0.0.1:5001,127.0.0.1:5002 --leader &

./bin/server --id srv2 --http 5001 --peers
127.0.0.1:5000,127.0.0.1:5002 &

./bin/server --id srv3 --http 5002 --peers
127.0.0.1:5000,127.0.0.1:5001 &

./bin/sensor --id INT01 --server 127.0.0.1:5000
```

- Use `tmux` to run multiple nodes; distinct port ranges per team.
-

8 Step-by-Step Timeline

Week	Tasks	Deliverable
Week 1 (Nov 13–19)	Implement sensor + single server (/ingest , /summary)	Basic aggregation + UI preview
Week 2 (Nov 20–26)	Add hash sharding + replication + failover	3-node cluster demo
Week 3 (Nov 27–Dec 2)	Run experiments, collect metrics, record demo	Final report + video

traffic_monitoring/

- |— CMakeLists.txt # (optional) if you prefer CMake
- |— Makefile # primary build (g++, -std=c++11)
- |— README.md # quick start + cluster runbook
- |— third_party/ # header-only libs vendored
 - | |— json.hpp # nlohmann/json (single header)
 - | |— httpLib.h # cpp-httpLib (single header)
- |— config/
 - | |— cluster.json # nodes + shard map + RF
 - | |— roads.json # optional: seed road list for sensors
 - | |— dashboard.json # UI refresh rate, endpoints
- |— include/
 - | |— types/
 - | | |— TrafficReport.h
 - | | |— RoadSnapshot.h
 - | | |— ShardTypes.h # Node, ShardInfo, ShardMap
 - | |— util/
 - | | |— Logger.h
 - | | |— Time.h

```

| | └─ JsonIO.h          # load/save JSON config/snapshots
| └─ net/
| | └─ HttpServer.h      # thin wrapper around httpplib::Server
| | └─ HttpClient.h      # thin wrapper around httpplib::Client
| └─ agg/
| | └─ Aggregator.h      # rolling windows, EWMA, classify
| └─ repl/
| | └─ Replication.h      # leader/follower API
| | └─ RaftLite.h        # msgs: heartbeat, append, vote
| └─ shard/
| | └─ ShardRouter.h      # hash(road)%P → shard → leader/follower
| | └─ ShardRegistry.h    # in-mem view of shard ownership
| └─ server/
| | └─ MainServer.h       # HTTP routes, wiring
| | └─ Status.h          # /status structs
| └─ sensor/
|   └─ SensorRunner.h     # configurable sensor loop
└─ src/
  └─ main_server.cpp       # parses flags, boots server
  └─ main_sensor.cpp       # launches N sensors (or 1) with params
  └─ types/
    └─ TrafficReport.cpp   # (usually header-only; keep for clarity)
    └─ RoadSnapshot.cpp
  └─ util/
    └─ Logger.cpp
    └─ Time.cpp
    └─ JsonIO.cpp
  └─ net/

```

```

| | └── HttpServer.cpp
| | └── HttpClient.cpp
| └── agg/
| | └── Aggregator.cpp
| └── repl/
| | └── Replication.cpp
| | └── RaftLite.cpp
| └── shard/
| | └── ShardRouter.cpp
| | └── ShardRegistry.cpp
| └── server/
|   └── MainServer.cpp
└── dashboard/
    ├── index.html      # simple map + JS fetch /summary
    ├── app.py          # (alt) Streamlit prototype
    └── static/
        └── map.js       # draws colored polylines (roads)
└── data/
    ├── logs/           # ingestion/replication logs (optional)
    └── snapshot.json    # periodic state dump (optional)
└── scripts/
    ├── run_local_3node.sh  # spawn srv1..srv3 + sensors
    ├── kill_leader.sh      # demo failover
    ├── collect_metrics.sh  # curl loops: latency/throughput
    └── khoury_runbook.sh    # tmux commands & port allocations
└── tests/
    ├── test_aggregator.cpp  # correctness of rolling windows
    └── test_shard_router.cpp # hash distribution determinism

```

└─ test_replication.cpp # happy path append/commit/apply